Mastering Spring Security -- Authentication, Authorization, and Enterprise Security Architecture Guide

---

Author: Senior Java Security Architect **Target Audience:** Java backend developers with 3+ years experience aiming for architect-level Spring Security expertise **Prerequisites:** Core Java, Spring Boot, REST APIs, basic security concepts

---

Table of Contents

1. Spring Security Fundamentals
2. Spring Security Architecture Internals
3. Complete Authentication Flow
4. Authentication Providers
5. UserDetails and UserDetailsService Deep Dive
6. Password Security and Encoding
7. JWT Authentication Architecture
8. Spring Security with OAuth2
9. Role-Based Access Control (RBAC)
10. Security Filter Chain Deep Dive
11. Session-Based vs Token-Based Authentication
12. Microservices Security Architecture
13. Security Best Practices
14. Production Security Challenges
15. Complete Spring Boot Security Project
16. Interview Questions (100+)
17. Hands-on Practice Exercises

Part 1: Spring Security Fundamentals

1.1 What is Spring Security?

Spring Security is a powerful, highly customizable authentication and access-control framework for Java applications. It is the de-facto standard for securing Spring-based applications.

Spring Security provides:

- **Authentication:** Verifying who you are (login)
- **Authorization:** Verifying what you can do (permissions)
- **Protection:** Against common security exploits (CSRF, XSS, session fixation)

1.2 Why Spring Security?

Need	How Spring Security Helps
User login	Multiple authentication mechanisms (form, JWT, OAuth2)
API protection	Secure REST endpoints with token-based auth
Role-based access	Fine-grained method and URL security
Password safety	Built-in BCrypt, Argon2 password encoders
Attack prevention	CSRF protection, CORS, headers security
Extensibility	Fully customizable filter chain

1.3 Common Security Threats

Threat	Description	Spring Security Defense
SQL Injection	Malicious SQL in input	JPA/Hibernate parameterized queries
Cross-Site Scripting (XSS)	Injecting scripts into pages	Response header security
CSRF	Forged requests from other sites	CSRF token validation
Session Hijacking	Stealing session cookies	Session fixation protection
Brute Force	Guessing passwords repeatedly	Account lockout, rate limiting

Credential Stuffing	Using leaked credentials	Password encoding, MFA
Man-in-the-Middle	Intercepting network traffic	HTTPS enforcement (HSTS)

1.4 Authentication vs Authorization

```
AUTHENTICATION (AuthN):                AUTHORIZATION (AuthZ):
"Who are you?"                         "What can you do?"

+----------+     +-----------+         +----------+     +-----------+
| Username | --> | Verify    |         | User     | --> | Check     |
| Password |     | Identity  |         | Role     |     | Permission|
+----------+     +-----------+         +----------+     +-----------+
                      |                                       |
              Valid/Invalid                           Allowed/Denied

Examples:                              Examples:
- Login with email + password          - ADMIN can delete users
- Login with Google (OAuth2)           - USER can view own profile
- Login with fingerprint               - MANAGER can approve orders
```

Part 2: Spring Security Architecture Internals

2.1 Architecture Diagram

```
HTTP Request
     |
     v
+-------------------------------------------------------------------+
|                   SECURITY FILTER CHAIN                           |
|                                                                    |
|  +-------------------+                                            |
|  | Security Context  |  Stores authenticated user info            |
|  | Persistence Filter|  (loads SecurityContext from session)       |
|  +-------------------+                                            |
|          |                                                         |
|  +-------------------+                                            |
|  | CSRF Filter       |  Validates CSRF token                      |
|  +-------------------+                                            |
|          |                                                         |
|  +-------------------+                                            |
|  | Authentication    |  UsernamePasswordAuthenticationFilter      |
|  | Filter            |  OR BearerTokenAuthenticationFilter        |
|  +-------------------+                                            |
|          |                                                         |
|          v                                                         |
|  +-------------------+     +------------------------+             |
|  | Authentication    | --> | Authentication         |             |
|  | Manager           |     | Provider               |             |
|  | (ProviderManager) |     | (DaoAuthProvider)      |             |
|  +-------------------+     +------------------------+             |
|                                     |                              |
|                            +--------+--------+                    |
|                            |                 |                    |
|                   +--------v-----+   +-------v--------+          |
|                   |UserDetails   |   |Password        |          |
|                   |Service       |   |Encoder         |          |
|                   |(loads user)  |   |(verifies pwd)  |          |
|                   +--------------+   +----------------+          |
|          |                                                         |
|  +-------------------+                                            |
|  | Exception         |  Handles AuthenticationException           |
|  | Translation Filter|  and AccessDeniedException                 |
|  +-------------------+                                            |
```

```
|           |                                                          |
|  +-------------------+                                              |
|  | Authorization     |  Checks roles/authorities                   |
|  | Filter            |  for the requested URL                     |
|  +-------------------+                                              |
+-------------------------------------------------------------------+
     |
     v
Controller (if authorized)
```

2.2 Core Components

SecurityFilterChain

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .csrf(csrf -> csrf.disable())  // Disable for REST APIs
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").authenticated()
            )
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

AuthenticationManager

What: Central interface for authentication. Receives an Authentication request and returns a fully authenticated object.

```java
@Bean
public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
```

AuthenticationProvider

What: Performs the actual authentication logic. The AuthenticationManager delegates to one or more providers.

SecurityContext and SecurityContextHolder

```java
// After authentication, the user's details are stored:
SecurityContext context = SecurityContextHolder.getContext();
Authentication auth = context.getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();

// Spring stores this per-thread using ThreadLocal
// Available throughout the request lifecycle
```

Part 3: Complete Authentication Flow

3.1 Step-by-Step Flow

```
Step 1: Client sends login request
        POST /api/auth/login
        { "email": "user@mail.com", "password": "secret123" }
              |
              v
Step 2: Request hits Security Filter Chain
        UsernamePasswordAuthenticationFilter intercepts
        Creates UsernamePasswordAuthenticationToken (unauthenticated)
              |
              v
Step 3: AuthenticationManager receives token
        Delegates to configured AuthenticationProvider(s)
              |
              v
Step 4: DaoAuthenticationProvider processes authentication
        Calls UserDetailsService.loadUserByUsername("user@mail.com")
              |
              v
Step 5: UserDetailsService loads user from database
        Returns UserDetails object with:
        - username, passwordHash, authorities, accountStatus
              |
              v
Step 6: Password validation
        PasswordEncoder.matches("secret123", "$2a$10$...")
        BCrypt compares raw password with stored hash
              |
              v
Step 7: If valid: Create authenticated token
        UsernamePasswordAuthenticationToken (authenticated = true)
        Contains: principal, authorities
              |
              v
Step 8: SecurityContextHolder stores authentication
        SecurityContextHolder.getContext().setAuthentication(authToken)
              |
              v
Step 9: Generate response (JWT token or session)
```

```
        Return JWT to client for stateless auth
        Or create HTTP session for stateful auth
```

3.2 Authentication Token States

```
// BEFORE authentication (unauthenticated):
UsernamePasswordAuthenticationToken token =
    new UsernamePasswordAuthenticationToken(email, password);
// token.isAuthenticated() == false
// token.getPrincipal() == "user@mail.com" (username)
// token.getCredentials() == "secret123" (password)

// AFTER authentication (authenticated):
UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(userDetails, null, authorities);
// authToken.isAuthenticated() == true
// authToken.getPrincipal() == UserDetails object
// authToken.getCredentials() == null (cleared for security)
// authToken.getAuthorities() == [ROLE_USER, ROLE_ADMIN]
```

Part 4: Authentication Providers

4.1 DaoAuthenticationProvider (Most Common)

What: Authenticates using a UserDetailsService and PasswordEncoder. **When:** Username/password authentication against a database.

```
@Bean
public AuthenticationProvider daoAuthProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService);
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}
```

4.2 JwtAuthenticationProvider (Custom)

What: Validates JWT tokens from request headers. **When:** Stateless REST API authentication.

```
// Custom JWT Authentication Filter
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired private JwtService jwtService;
    @Autowired private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        String username = jwtService.extractUsername(token);
```

```
        if (username != null && SecurityContextHolder.getContext()
                .getAuthentication() == null) {

            UserDetails userDetails = userDetailsService
                .loadUserByUsername(username);

            if (jwtService.isTokenValid(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(
                    new WebAuthenticationDetailsSource()
                        .buildDetails(request));
                SecurityContextHolder.getContext()
                    .setAuthentication(authToken);
            }
        }
        chain.doFilter(request, response);
    }
}
```

4.3 OAuth2 Authentication Provider

What: Authenticates using external OAuth2 providers (Google, GitHub). **When:** "Login with Google" or SSO.

4.4 LDAP Authentication Provider

What: Authenticates against an LDAP directory (Active Directory). **When:** Enterprise environments with centralized user directories.

4.5 RememberMe Authentication Provider

What: Authenticates using a persistent remember-me token. **When:** "Keep me logged in" functionality.

4.6 Anonymous Authentication Provider

What: Creates an anonymous authentication token for unauthenticated requests. **When:** Allowing certain pages to be publicly accessible while still having a SecurityContext.

4.7 Pre-Authenticated Authentication Provider

What: Trusts authentication already done by an external system (API Gateway, SSO). **When:** Behind a reverse proxy that has already validated the user.

Provider	Use Case	Authentication Source
DaoAuthenticationProvider	Database login	UserDetailsService + PasswordEncoder
Custom JwtAuthProvider	REST API auth	JWT token in Authorization header
OAuth2AuthProvider	Social login	Google, GitHub, Microsoft
LdapAuthProvider	Enterprise SSO	Active Directory / LDAP
RememberMeProvider	Persistent login	Cookie token
AnonymousProvider	Public pages	Auto-generated anonymous token
PreAuthProvider	Gateway-authenticated	Trusted headers from proxy

---

Part 5: UserDetails and UserDetailsService

5.1 UserDetails Interface

```
public interface UserDetails extends Serializable {
    Collection<? extends GrantedAuthority> getAuthorities();
    String getPassword();
    String getUsername();
    boolean isAccountNonExpired();
    boolean isAccountNonLocked();
    boolean isCredentialsNonExpired();
    boolean isEnabled();
}
```

5.2 Custom UserDetails Implementation

```
@Getter
public class CustomUserDetails implements UserDetails {

    private final Long id;
    private final String email;
    private final String password;
    private final String fullName;
    private final Collection<? extends GrantedAuthority> authorities;
    private final boolean active;
    private final boolean locked;

    public CustomUserDetails(User user) {
        this.id = user.getId();
        this.email = user.getEmail();
        this.password = user.getPasswordHash();
        this.fullName = user.getFirstName() + " " + user.getLastName();
        this.authorities = user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toList());
        this.active = user.isActive();
        this.locked = user.isLocked();
    }
```

```
    @Override public String getUsername() { return email; }
    @Override public String getPassword() { return password; }
    @Override public boolean isEnabled() { return active; }
    @Override public boolean isAccountNonLocked() { return !locked; }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }
}
```

5.3 Custom UserDetailsService

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String email)
            throws UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found with email: " + email));

        if (!user.isActive()) {
            throw new DisabledException("Account is deactivated");
        }
        if (user.isLocked()) {
            throw new LockedException("Account is locked");
        }

        return new CustomUserDetails(user);
    }
}
```

Part 6: Password Security and Encoding

6.1 Why Password Encoding?

```
NEVER store passwords as plain text!

Plain text:   password123          (compromised = instant access)
MD5 hash:     482c811da5d5b4...   (weak, rainbow table attacks)
SHA-256:      ef92b778bafe77...   (weak, no salt)
BCrypt:       $2a$10$N9qo8uL...  (strong, salted, slow by design)
```

6.2 Password Encoders

Encoder	Strength	Speed	Use In Production?
`NoOpPasswordEncoder`	None	Instant	NEVER (testing only)
`BCryptPasswordEncoder`	Strong	~100ms	Yes (recommended)
`SCryptPasswordEncoder`	Very strong	~200ms	Yes (memory-hard)
`Argon2PasswordEncoder`	Strongest	~300ms	Yes (latest standard)

6.3 BCrypt (Recommended)

```java
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // Strength 12 (2^12 iterations)
}

// Registration: encode password before saving
String rawPassword = "secret123";
String encoded = passwordEncoder.encode(rawPassword);
// $2a$12$LJ3m4ys1jN6cVKkWJPz2U.Bj6YOyI1L5lq0A9X5JVfKq3O7/HZOW.
// Each encode() call produces DIFFERENT output (random salt)

// Login: verify password
```

```
boolean matches = passwordEncoder.matches("secret123", encoded);
// true (compares correctly despite different hashes due to embedded salt)
```

6.4 How BCrypt Works

```
Input: "secret123"

Step 1: Generate random salt (22 characters)
        Salt: LJ3m4ys1jN6cVKkWJPz2U.

Step 2: Hash password + salt using Blowfish cipher
        Run 2^12 = 4096 iterations (slow by design!)

Step 3: Combine: $2a$12$<salt><hash>
        $2a$12$LJ3m4ys1jN6cVKkWJPz2U.Bj6YOyI1L5lq0A9X5JVfKq3O7/HZOW.

Why BCrypt is secure:
- Salt prevents rainbow table attacks
- Slow hashing prevents brute force (4096 iterations)
- Cost factor can be increased as hardware gets faster
```

6.5 Delegating Password Encoder

```
// Supports multiple encoders for migration scenarios
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    // Stores format: {bcrypt}$2a$10$...
    // Supports: {bcrypt}, {scrypt}, {argon2}, {noop}
    // Default encoding: BCrypt
}
```

Part 7: JWT Authentication Architecture

7.1 What is JWT?

JSON Web Token (JWT) is a compact, self-contained token used for securely transmitting information between parties. It contains all user information needed for authentication -- no server-side session storage required.

Why JWT Over Session-Based Authentication

Session-Based	JWT
Server stores session in memory/Redis	Token stored on client (stateless server)
Requires sticky sessions for scaling	Any server can validate the token
Difficult for mobile apps	Works with any HTTP client
CSRF vulnerable (cookie-based)	Not CSRF vulnerable (header-based)
Easy to revoke (delete session)	Hard to revoke (needs blacklist)

When to Use JWT

- **REST APIs** consumed by SPAs, mobile apps, or other services
- **Microservices** -- stateless authentication without shared session store
- When you need **self-contained tokens** that carry user claims

When NOT to Use JWT

- **Traditional server-rendered web apps** -- sessions are simpler
- When **instant revocation** is critical -- JWT tokens are valid until expiry
- For **very sensitive operations** -- combine JWT with additional verification

Security Best Practices

- Use short-lived **access tokens** (15-30 minutes) + longer **refresh tokens** (7 days)
- Store tokens in **httpOnly cookies** (not localStorage) for web apps
- Always use **HTTPS**
- Rotate signing keys periodically

7.2 JWT Structure

```
Header.Payload.Signature

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyQG1haWwuY29tIiwicm9sZXMi...
|_______Header________|._________________Payload_________________|.Sig

HEADER (Algorithm + Type):
{
  "alg": "HS256",
  "typ": "JWT"
}

PAYLOAD (Claims - user data):
{
  "sub": "user@mail.com",        // Subject (username)
  "roles": ["ROLE_USER"],         // User roles
  "iat": 1709812345,              // Issued at
  "exp": 1709898745               // Expiration (24 hours)
}

SIGNATURE:
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```

7.3 JWT Authentication Flow

```
1. LOGIN:
   Client                    Auth Server
     |  POST /auth/login       |
     |  { email, password }    |
     |------------------------>|
     |                         | Validate credentials
     |                         | Generate JWT token
     |  { accessToken,         |
     |    refreshToken }       |
     |<------------------------|

2. API CALLS:
   Client                    Resource Server
     |  GET /api/orders        |
     |  Authorization:         |
     |  Bearer eyJhbGci...     |
     |------------------------>|
```

```
     |                          | Extract JWT from header
     |                          | Validate signature
     |                          | Check expiration
     |                          | Extract user info
     |                          | Set SecurityContext
     |  { orders data }         |
     |<-------------------------|

3. TOKEN REFRESH:
   Client                     Auth Server
     |  POST /auth/refresh      |
     |  { refreshToken }        |
     |------------------------->|
     |                          | Validate refresh token
     |                          | Issue new access token
     |  { newAccessToken }      |
     |<-------------------------|
```

7.4 JWT Service Implementation

```java
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.access-token-expiration}")
    private long accessTokenExpiry;  // 15 minutes

    @Value("${jwt.refresh-token-expiration}")
    private long refreshTokenExpiry;  // 7 days

    public String generateAccessToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("roles", userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList()));
        return buildToken(claims, userDetails.getUsername(),
accessTokenExpiry);
    }

    public String generateRefreshToken(UserDetails userDetails) {
        return buildToken(new HashMap<>(), userDetails.getUsername(),
                          refreshTokenExpiry);
```

```java
    }

    private String buildToken(Map<String, Object> claims,
                              String subject, long expiry) {
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(subject)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiry))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername())
            && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }

    private <T> T extractClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
        return resolver.apply(claims);
    }

    private Key getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

7.5 Auth Controller

```java
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;
    private final UserService userService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest
request) {
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getEmail(), request.getPassword()));

        UserDetails user = userDetailsService
            .loadUserByUsername(request.getEmail());
        String accessToken = jwtService.generateAccessToken(user);
        String refreshToken = jwtService.generateRefreshToken(user);

        return ResponseEntity.ok(new AuthResponse(accessToken, refreshToken));
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(
            @RequestBody @Valid RegisterRequest request) {
        UserDetails user = userService.register(request);
        String accessToken = jwtService.generateAccessToken(user);
        String refreshToken = jwtService.generateRefreshToken(user);
        return ResponseEntity.status(201)
            .body(new AuthResponse(accessToken, refreshToken));
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refresh(
            @RequestBody RefreshRequest request) {
        String username =
jwtService.extractUsername(request.getRefreshToken());
        UserDetails user = userDetailsService.loadUserByUsername(username);

        if (!jwtService.isTokenValid(request.getRefreshToken(), user)) {
            throw new InvalidTokenException("Invalid refresh token");
        }
```

```
        String newAccessToken = jwtService.generateAccessToken(user);
        return ResponseEntity.ok(
            new AuthResponse(newAccessToken, request.getRefreshToken()));
    }
}
```

Part 8: Spring Security with OAuth2

What is OAuth2?

OAuth 2.0 is an **authorization framework** that allows third-party applications to access a user's data without knowing their password. It's the protocol behind "Login with Google/GitHub/Facebook" buttons.

Why OAuth2 Exists

Before OAuth2, applications asked users for their Google/Facebook password directly. This was a security nightmare. OAuth2 solves this by giving the application a **limited-scope access token** instead of the user's credentials.

Key Insight: Authentication vs Authorization

- **OAuth2** is for **authorization** (what can you access?)
- **OpenID Connect (OIDC)** is for **authentication** (who are you?) -- built on top of OAuth2
- In practice, "Login with Google" uses OIDC, which is OAuth2 + user identity

When to Use OAuth2

- **Social login** -- "Login with Google/GitHub"
- **Service-to-service** communication (Client Credentials flow)
- When you need a **centralized identity provider** (Keycloak, Auth0)

When NOT to Use OAuth2

- Simple applications with their own login form -- use username/password + JWT
- Internal tools with limited users -- use basic auth or LDAP

8.1 OAuth2 Concepts

```
+----------+     +----------+     +----------+     +----------+
| Resource |     | Client   |     | Auth     |     | Resource |
| Owner    |     | App      |     | Server   |     | Server   |
| (User)   |     | (Your    |     | (Google/ |     | (Your    |
|          |     |  App)    |     |  GitHub) |     |  API)    |
+----------+     +----------+     +----------+     +----------+

Resource Owner: The user who owns the data
Client App: Your application requesting access
```

```
Authorization Server: Issues tokens (Google, GitHub, Keycloak)
Resource Server: Hosts protected resources (your API)
```

8.2 OAuth2 Flows

Authorization Code Flow (Most Secure, Web Apps)

```
1. User clicks "Login with Google"
2. App redirects to Google's authorization page
3. User logs in to Google, grants permission
4. Google redirects back with authorization code
5. App exchanges code for access token (server-side)
6. App uses access token to get user info
7. App creates local session/JWT

    User             Your App              Google
     |  Click Login     |                    |
     |----------------->|                    |
     |                  |  Redirect to Google |
     |                  |-------------------->|
     |                  |                    | Login page
     |<-------------------------------------|
     |  Enter credentials                    |
     |-------------------------------------->|
     |                  |  Redirect + code    |
     |                  |<--------------------|
     |                  |  Exchange code      |
     |                  |-------------------->|
     |                  |  Access token       |
     |                  |<--------------------|
     |                  |  Get user info      |
     |                  |-------------------->|
     |                  |  User data          |
     |                  |<--------------------|
     |  Logged in!      |                    |
     |<-----------------|                    |
```

Client Credentials Flow (Service-to-Service)

```
Service A                    Auth Server (Keycloak)
   |  POST /token              |
   |  grant_type=client_creds  |
```

```
   |  client_id + secret       |
   |-------------------------->|
   |  { access_token }         |
   |<--------------------------|
   |                           |
   |  GET /api/data            |  Service B
   |  Authorization: Bearer... |  (Resource Server)
   |------------------------------------------->|
   |  { data }                                  |
   |<-------------------------------------------|
```

8.3 Spring Boot OAuth2 Configuration

```yaml
# application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: openid, profile, email
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
            scope: read:user, user:email
```

```java
@Configuration
@EnableWebSecurity
public class OAuth2SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth -> oauth
                .loginPage("/login")
```

```
                .userInfoEndpoint(userInfo ->
                    userInfo.userService(customOAuth2UserService))
                .successHandler(oAuth2SuccessHandler)
            )
            .build();
    }
}

@Service
public class CustomOAuth2UserService
        extends DefaultOAuth2UserService {

    @Autowired private UserRepository userRepository;

    @Override
    public OAuth2User loadUser(OAuth2UserRequest request)
            throws OAuth2AuthenticationException {
        OAuth2User oAuth2User = super.loadUser(request);
        String email = oAuth2User.getAttribute("email");
        String name = oAuth2User.getAttribute("name");
        String provider = request.getClientRegistration()
            .getRegistrationId(); // "google" or "github"

        // Find or create local user
        User user = userRepository.findByEmail(email)
            .orElseGet(() -> {
                User newUser = new User();
                newUser.setEmail(email);
                newUser.setName(name);
                newUser.setProvider(provider);
                newUser.setRole(Role.USER);
                return userRepository.save(newUser);
            });

        return new CustomOAuth2User(oAuth2User, user);
    }
}
```

Part 9: Role-Based Access Control (RBAC)

What is RBAC?

Role-Based Access Control assigns permissions through **roles** rather than directly to users. A user is assigned roles (ADMIN, USER, MANAGER), and each role has a set of permissions.

Why RBAC Over Direct Permissions

```
Direct Permissions (hard to manage):     RBAC (scalable):
  User1 -> [read, write, delete]          User1 -> ADMIN role -> [read, write,
delete]
  User2 -> [read, write, delete]          User2 -> ADMIN role -> [read, write,
delete]
  User3 -> [read]                         User3 -> USER role  -> [read]

  To add a new permission:               To add a new permission:
  Update ALL admin users individually    Update the ADMIN role once
```

Design Pattern: Two-Level Authorization

Spring Security supports both coarse-grained (role) and fine-grained (authority) access control:

- **URL-based security** (coarse) -- configured in `SecurityFilterChain`
- **Method-level security** (fine) -- `@PreAuthorize`, `@Secured` annotations

9.1 Roles vs Authorities

```
ROLE: A group of permissions (coarse-grained)
  - ROLE_USER
  - ROLE_ADMIN
  - ROLE_MANAGER

AUTHORITY: A specific permission (fine-grained)
  - user:read
  - user:write
  - order:delete
  - report:export

Hierarchy:
```

```
ROLE_ADMIN -> [user:read, user:write, user:delete,
               order:read, order:write, order:delete,
               report:read, report:export]
ROLE_MANAGER -> [user:read, order:read, order:write,
                 report:read, report:export]
ROLE_USER -> [user:read, order:read]
```

9.2 URL-Based Security

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            // Public endpoints
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()

            // Role-based
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/reports/**").hasAnyRole("ADMIN", "MANAGER")

            // Authority-based
            .requestMatchers(HttpMethod.DELETE, "/api/users/**")
                .hasAuthority("user:delete")
            .requestMatchers(HttpMethod.POST, "/api/orders/**")
                .hasAuthority("order:write")

            // Everything else requires authentication
            .anyRequest().authenticated()
        )
        .build();
}
```

9.3 Method-Level Security

```java
@Configuration
@EnableMethodSecurity  // Enables @PreAuthorize, @PostAuthorize, @Secured
public class MethodSecurityConfig { }

@Service
```

```java
public class UserService {

    // @PreAuthorize: checked BEFORE method execution
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long userId) {
        userRepository.deleteById(userId);
    }

    // Access method parameters
    @PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
    public User getUser(@Param("userId") Long userId) {
        return userRepository.findById(userId).orElseThrow();
    }

    // @PostAuthorize: checked AFTER method execution
    @PostAuthorize("returnObject.email == authentication.name")
    public User getMyProfile(Long userId) {
        return userRepository.findById(userId).orElseThrow();
    }

    // @Secured: simpler (no SpEL)
    @Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    // @RolesAllowed (JSR-250)
    @RolesAllowed("ADMIN")
    public void resetPassword(Long userId) {
        // admin operation
    }
}
```

9.4 Database Schema for RBAC

```cpp
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE roles (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```sql
    name VARCHAR(50) UNIQUE NOT NULL  -- ADMIN, USER, MANAGER
);

CREATE TABLE permissions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL  -- user:read, order:write
);

CREATE TABLE user_roles (
    user_id BIGINT NOT NULL,
    role_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (role_id) REFERENCES roles(id)
);

CREATE TABLE role_permissions (
    role_id BIGINT NOT NULL,
    permission_id BIGINT NOT NULL,
    PRIMARY KEY (role_id, permission_id),
    FOREIGN KEY (role_id) REFERENCES roles(id),
    FOREIGN KEY (permission_id) REFERENCES permissions(id)
);
```

Part 10: Security Filter Chain Deep Dive

What is the Security Filter Chain?

The **Security Filter Chain** is a series of servlet filters that process every HTTP request before it reaches your controller. Each filter handles one specific security concern (authentication, CSRF, session management, authorization).

Why Understanding the Filter Chain Matters

- To add **custom authentication** (JWT filter), you must know WHERE to insert it
- To debug security issues, you need to understand WHICH filter is rejecting the request
- To customize error responses (401/403), you need to know which filter handles them

Key Insight

```
The order of filters matters!
  - Authentication filters run BEFORE authorization filters
  - If a request passes all filters, it's both authenticated AND authorized
  - Your custom JWT filter should run BEFORE
UsernamePasswordAuthenticationFilter
```

10.1 Default Filter Execution Order

```
HTTP Request
     |
     v
+-------------------------------------------------------+
|                 SECURITY FILTER CHAIN                  |
|                                                         |
| 1. SecurityContextPersistenceFilter                    |
|    Load SecurityContext from session/storage            |
|                    |                                    |
| 2. HeaderWriterFilter                                  |
|    Add security headers (X-Frame-Options, etc.)        |
|                    |                                    |
| 3. CsrfFilter                                         |
|    Validate CSRF token for state-changing requests     |
|                    |                                    |
| 4. LogoutFilter                                        |
|    Handle /logout requests                             |
```

```
|                    |                                    |
| 5. UsernamePasswordAuthenticationFilter                 |
|    Handle form login POST /login                        |
|    (or custom JWT filter inserted here)                 |
|                    |                                    |
| 6. BasicAuthenticationFilter                            |
|    Handle HTTP Basic auth (Authorization: Basic ...)    |
|                    |                                    |
| 7. BearerTokenAuthenticationFilter                      |
|    Handle Bearer token (Authorization: Bearer ...)      |
|                    |                                    |
| 8. RequestCacheAwareFilter                              |
|    Restore previously cached request                    |
|                    |                                    |
| 9. SecurityContextHolderAwareRequestFilter              |
|    Wrap request with security methods                   |
|                    |                                    |
| 10. AnonymousAuthenticationFilter                       |
|     Create anonymous auth if not authenticated          |
|                    |                                    |
| 11. SessionManagementFilter                             |
|     Manage session creation policy                      |
|                    |                                    |
| 12. ExceptionTranslationFilter                          |
|     Catch and translate security exceptions             |
|     AuthenticationException -> 401                      |
|     AccessDeniedException -> 403                        |
|                    |                                    |
| 13. AuthorizationFilter                                 |
|     Check if user has required role/authority           |
|     for the requested URL                               |
+---------------------------------------------------------+
     |
     v (if authorized)
Controller handles request
```

10.2 Custom Filter Registration

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        // ... configuration ...

        // Add JWT filter BEFORE the default authentication filter
```

```
        .addFilterBefore(jwtAuthFilter,
            UsernamePasswordAuthenticationFilter.class)

        // Add rate limiting filter BEFORE everything
        .addFilterBefore(rateLimitFilter,
            SecurityContextPersistenceFilter.class)

        // Add logging filter AFTER authentication
        .addFilterAfter(requestLoggingFilter,
            AuthorizationFilter.class)

        .build();
}
```

10.3 ExceptionTranslationFilter

```
// Customizing authentication and access denied responses
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint((request, response, exception) -> {
                response.setStatus(401);
                response.setContentType("application/json");
                response.getWriter().write(
                    "{\"error\": \"Authentication required\", " +
                    "\"message\": \"" + exception.getMessage() + "\"}");
            })
            .accessDeniedHandler((request, response, exception) -> {
                response.setStatus(403);
                response.setContentType("application/json");
                response.getWriter().write(
                    "{\"error\": \"Access denied\", " +
                    "\"message\": \"Insufficient permissions\"}");
            })
        )
        .build();
}
```

Part 11: Session-Based vs Token-Based Authentication

Why This Comparison Matters

Choosing between session-based and token-based authentication is one of the most important architectural decisions for your application. It affects scalability, security, and how you handle authentication across your entire system.

11.1 Comparison

```
SESSION-BASED:                          TOKEN-BASED (JWT):

Client                Server            Client                Server
  |  Login              |               |  Login              |
  |-------------------->|               |-------------------->|
  |  Set-Cookie:        |               |  { accessToken,     |
  |  JSESSIONID=abc123  |               |    refreshToken }   |
  |<--------------------|               |<--------------------|
  |                      |               |                      |
  |  GET /api/data      |               |  GET /api/data      |
  |  Cookie: abc123     |               |  Authorization:     |
  |-------------------->|               |  Bearer eyJhbG...   |
  |  Server looks up    |               |-------------------->|
  |  session in memory  |               |  Server validates   |
  |  or Redis           |               |  token signature    |
  |  { data }           |               |  (no DB lookup)     |
  |<--------------------|               |  { data }           |
                                        |<--------------------|
```

Aspect	Session-Based	Token-Based (JWT)
Storage	Server-side (memory/Redis)	Client-side (localStorage/cookie)
Scalability	Requires sticky sessions or shared session store	Stateless, any server can handle
Mobile support	Cookie management, complex	Straightforward, header-based
Revocation	Easy (delete session)	Hard (need blacklist or short expiry)
Performance	Session lookup per request	Signature verification (no DB hit)

Size	Small cookie (~32 bytes)	Larger token (~500+ bytes)
CSRF risk	Vulnerable (cookie-based)	Not vulnerable (header-based)
Best for	Traditional web apps (MVC)	REST APIs, microservices, SPAs

11.2 Spring Security Configuration

```java
// SESSION-BASED (default)
@Bean
public SecurityFilterChain sessionConfig(HttpSecurity http) throws Exception {
    return http
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                   .maximumSessions(1)  // Only 1 active session per user
                   .expiredUrl("/login?expired"))
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/dashboard"))
        .build();
}

// TOKEN-BASED (stateless)
@Bean
public SecurityFilterChain tokenConfig(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())  // No CSRF needed for stateless
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtFilter,
            UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Part 12: Microservices Security Architecture

12.1 Security Architecture

```
+-------+     +------------+     +----------+     +----------+
| Client| --> | API        | --> | Auth     | --> | User     |
| (SPA/ |     | Gateway    |     | Service  |     | DB       |
|  App) |     | (routes +  |     | (JWT     |     |          |
|       |     |  JWT valid)|     |  issuer) |     |          |
+-------+     +-----+------+     +----------+     +----------+
                    |
         +----------+----------+
         |          |          |
    +----v---+ +----v---+ +----v---+
    | Order  | | User   | | Payment|
    | Service| | Service| | Service|
    | (JWT   | | (JWT   | | (JWT   |
    |  valid)| |  valid)| |  valid)|
    +--------+ +--------+ +--------+
```

12.2 How It Works

```
Step 1: Client logs in via Auth Service
        POST /api/auth/login -> Auth Service validates credentials
        Returns JWT: { accessToken, refreshToken }

Step 2: Client sends API request with JWT
        GET /api/orders
        Authorization: Bearer eyJhbGci...

Step 3: API Gateway intercepts
        - Extracts JWT from header
        - Validates signature (does NOT call Auth Service)
        - Checks expiration
        - If valid: forwards request with user info headers
        - If invalid: returns 401

Step 4: Downstream service receives request
        - Reads user info from headers (set by Gateway)
```

```
        - OR validates JWT again (defense in depth)
        - Authorizes based on roles/permissions
```

12.3 API Gateway JWT Filter

```java
@Component
public class JwtGatewayFilter implements GatewayFilter {

    @Autowired private JwtService jwtService;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                             GatewayFilterChain chain) {
        String authHeader = exchange.getRequest()
            .getHeaders().getFirst("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        try {
            String username = jwtService.extractUsername(token);
            List<String> roles = jwtService.extractRoles(token);

            // Forward user info to downstream services
            ServerHttpRequest modifiedRequest = exchange.getRequest()
                .mutate()
                .header("X-User-Id", username)
                .header("X-User-Roles", String.join(",", roles))
                .build();

            return chain.filter(
                exchange.mutate().request(modifiedRequest).build());
        } catch (JwtException e) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }
}
```

12.4 Inter-Service Security

```
Service-to-Service Communication:

Option 1: Propagate user JWT
  Order Service -> Payment Service
  Forwards the original user JWT
  Payment Service validates the same token

Option 2: Service account JWT (Client Credentials)
  Order Service authenticates with its own credentials
  Gets a service-level JWT from Auth Service
  Uses service JWT to call Payment Service

Option 3: mTLS (Mutual TLS)
  Both services present certificates
  Verified at transport layer
  Used in service mesh (Istio)
```

```java
// Feign client that propagates JWT from the current request
@FeignClient(name = "payment-service",
             configuration = FeignAuthConfig.class)
public interface PaymentClient {
    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);
}

@Configuration
public class FeignAuthConfig {
    @Bean
    public RequestInterceptor authInterceptor() {
        return requestTemplate -> {
            // Get JWT from current SecurityContext
            ServletRequestAttributes attrs =
                (ServletRequestAttributes) RequestContextHolder
                    .getRequestAttributes();
            if (attrs != null) {
                String authHeader = attrs.getRequest()
                    .getHeader("Authorization");
                if (authHeader != null) {
                    requestTemplate.header("Authorization", authHeader);
                }
            }
        };
```

```
    }
}
```

Part 13: Security Best Practices

13.1 Secure Password Storage

```
// ALWAYS use BCrypt or Argon2
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}

// NEVER store plain text passwords
// NEVER use MD5 or SHA for passwords
// NEVER write your own hashing algorithm
```

13.2 CSRF Protection

```
// For REST APIs (stateless, JWT): DISABLE CSRF
// JWT in Authorization header is not auto-sent by browser -> no CSRF risk
http.csrf(csrf -> csrf.disable());

// For traditional web apps (session-based): KEEP CSRF ENABLED
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()));

// CSRF token is included in forms:
// <input type="hidden" name="_csrf" value="${_csrf.token}"/>
```

13.3 CORS Configuration

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of(
        "https://myapp.com",
        "https://admin.myapp.com"
```

```
    ));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}

// In SecurityFilterChain:
http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
```

13.4 Token Expiration Strategy

```
Access Token:  Short-lived (15 minutes)
  - Used for API authentication
  - If stolen, limited damage window
  - Client refreshes when expired

Refresh Token: Long-lived (7 days)
  - Used ONLY to get new access tokens
  - Stored securely (HttpOnly cookie)
  - Rotated on use (old one invalidated)
  - Revocable (stored in DB)

Token Rotation:
  1. Access token expires (15 min)
  2. Client sends refresh token to /auth/refresh
  3. Server issues new access token + new refresh token
  4. Old refresh token is invalidated
  5. If old refresh token is reused -> revoke ALL tokens (security alert)
```

13.5 Rate Limiting

```
@Component
public class RateLimitFilter extends OncePerRequestFilter {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();
```

```java
    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String clientIp = request.getRemoteAddr();
        Bucket bucket = buckets.computeIfAbsent(clientIp,
            k -> Bucket.builder()
                .addLimit(Bandwidth.classic(100,
                    Refill.intervally(100, Duration.ofMinutes(1))))
                .build());

        if (bucket.tryConsume(1)) {
            chain.doFilter(request, response);
        } else {
            response.setStatus(429);
            response.getWriter().write(
                "{\"error\": \"Too many requests\"}");
        }
    }
}
```

13.6 Security Headers

```java
http.headers(headers -> headers
    .frameOptions(frame -> frame.deny())                    // Clickjacking
    .contentTypeOptions(content -> content.disable())       // MIME sniffing
    .httpStrictTransportSecurity(hsts -> hsts               // HTTPS only
        .includeSubDomains(true)
        .maxAgeInSeconds(31536000))
    .xssProtection(xss -> xss.headerValue(
        XXssProtectionHeaderWriter.HeaderValue.ENABLED_MODE_BLOCK))
);
```

Part 14: Production Security Challenges

14.1 Token Leaks

Cause: JWT stored in localStorage (accessible via XSS), leaked in URLs, or logged in server logs.

Mitigation:

Strategy	How
Short expiry	Access token = 15 min. Limits damage.
HttpOnly cookies	Store tokens in HttpOnly cookies (no JS access)
Token blacklist	Maintain a blacklist in Redis for revoked tokens
Log sanitization	Never log authorization headers

```java
// Token blacklist check
@Component
public class TokenBlacklistService {
    @Autowired private RedisTemplate<String, String> redis;

    public void blacklist(String token, long expiryMs) {
        redis.opsForValue().set("blacklist:" + token, "revoked",
            Duration.ofMillis(expiryMs));
    }

    public boolean isBlacklisted(String token) {
        return Boolean.TRUE.equals(redis.hasKey("blacklist:" + token));
    }
}
```

14.2 Session Hijacking

Cause: Attacker steals session cookie via XSS or network sniffing.

Mitigation:

- Use HTTPS everywhere (HSTS header)
- Set cookies as HttpOnly + Secure + SameSite=Strict

- Session fixation protection (regenerate session ID on login)
- Bind session to IP/User-Agent

```
http.sessionManagement(session -> session
    .sessionFixation().migrateSession()  // New session ID on login
    .maximumSessions(1)                   // One session per user
    .maxSessionsPreventsLogin(true));      // Block new session if active
```

14.3 Replay Attacks

Cause: Attacker intercepts and resends a valid request.

Mitigation:

- Short-lived access tokens
- Include timestamp and nonce in requests
- Idempotency keys for critical operations
- Detect and block duplicate request IDs

14.4 Credential Stuffing

Cause: Attackers use leaked username/password lists from other sites.

Mitigation:

Strategy	Implementation
Rate limiting	Max 5 failed logins per IP per minute
Account lockout	Lock after 5 failed attempts, unlock after 30 min
CAPTCHA	After 3 failed attempts
MFA	Require second factor for login
Password policy	Minimum 8 chars, check against common password list

```
@Service
public class LoginAttemptService {
    private final LoadingCache<String, Integer> attemptsCache;

    public LoginAttemptService() {
```

```
        attemptsCache = CacheBuilder.newBuilder()
            .expireAfterWrite(30, TimeUnit.MINUTES)
            .build(CacheLoader.from(() -> 0));
    }

    public void loginFailed(String key) {
        int attempts = attemptsCache.getUnchecked(key);
        attemptsCache.put(key, attempts + 1);
    }

    public boolean isBlocked(String key) {
        return attemptsCache.getUnchecked(key) >= 5;
    }

    public void loginSucceeded(String key) {
        attemptsCache.invalidate(key);
    }
}
```

14.5 Security Incident Response

```
1. DETECT: Monitor for anomalies
   - Failed login spikes
   - Unusual API patterns
   - Unauthorized access attempts

2. CONTAIN: Limit damage
   - Revoke compromised tokens
   - Lock affected accounts
   - Block suspicious IPs

3. INVESTIGATE: Find root cause
   - Review audit logs
   - Trace request chain
   - Identify vulnerability

4. REMEDIATE: Fix the issue
   - Patch vulnerability
   - Rotate secrets
   - Force password resets

5. RECOVER: Return to normal
   - Unlock accounts
```

```
    - Restore services
    - Notify affected users
```

Part 15: Complete Spring Boot Security Project

15.1 Project Structure

```
secure-app/
|
+-- src/main/java/com/app/
|   |
|   +-- SecureAppApplication.java
|   |
|   +-- controller/
|   |   +-- AuthController.java          // Login, Register, Refresh
|   |   +-- UserController.java          // User CRUD
|   |   +-- AdminController.java         // Admin operations
|   |
|   +-- service/
|   |   +-- AuthService.java             // Authentication logic
|   |   +-- UserService.java             // User business logic
|   |   +-- TokenService.java            // Token management
|   |
|   +-- repository/
|   |   +-- UserRepository.java          // JPA User repository
|   |   +-- RoleRepository.java          // JPA Role repository
|   |   +-- RefreshTokenRepository.java  // Refresh token storage
|   |
|   +-- entity/
|   |   +-- User.java                    // User JPA entity
|   |   +-- Role.java                    // Role entity
|   |   +-- Permission.java             // Permission entity
|   |   +-- RefreshToken.java           // Refresh token entity
|   |
|   +-- security/
|   |   +-- SecurityConfig.java          // Security filter chain config
|   |   +-- JwtAuthenticationFilter.java // JWT validation filter
|   |   +-- JwtService.java             // JWT generation/validation
|   |   +-- CustomUserDetails.java      // UserDetails implementation
|   |   +-- CustomUserDetailsService.java// Load user from DB
|   |
|   +-- config/
|   |   +-- CorsConfig.java             // CORS settings
|   |   +-- OpenApiConfig.java          // Swagger security config
|   |
|   +-- dto/
```

```
|   |   +-- request/
|   |   |   +-- LoginRequest.java
|   |   |   +-- RegisterRequest.java
|   |   |   +-- RefreshTokenRequest.java
|   |   +-- response/
|   |       +-- AuthResponse.java
|   |       +-- UserResponse.java
|   |       +-- ApiResponse.java
|   |
|   +-- exception/
|   |   +-- GlobalExceptionHandler.java  // @ControllerAdvice
|   |   +-- InvalidTokenException.java
|   |   +-- UserAlreadyExistsException.java
|   |
|   +-- mapper/
|       +-- UserMapper.java              // Entity <-> DTO mapping
|
+-- src/main/resources/
|   +-- application.yml
|   +-- application-dev.yml
|   +-- application-prod.yml
|
+-- src/test/java/com/app/
|   +-- controller/
|   |   +-- AuthControllerTest.java
|   +-- security/
|       +-- JwtServiceTest.java
|
+-- pom.xml
```

15.2 Key Component Implementations

Entity: User

```java
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;
```

```
    @Column(nullable = false)
    private String passwordHash;

    private String firstName;
    private String lastName;
    private boolean active = true;
    private boolean locked = false;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();

    @Column(updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() { createdAt = LocalDateTime.now(); }
}
```

Security Configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;
    private final CustomUserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfig()))
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/users/me").authenticated()
                .anyRequest().authenticated()
```

```
            )
            .authenticationProvider(authProvider())
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint((req, res, e) -> {
                    res.setStatus(401);
                    res.setContentType("application/json");
                    res.getWriter().write("{\"error\":\"Unauthorized\"}");
                })
                .accessDeniedHandler((req, res, e) -> {
                    res.setStatus(403);
                    res.setContentType("application/json");
                    res.getWriter().write("{\"error\":\"Forbidden\"}");
                })
            )
            .build();
    }

    @Bean
    public AuthenticationProvider authProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public AuthenticationManager authManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

Application Configuration

```
# application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/secureapp
```

```yaml
    username: app_user
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect

jwt:
  secret: ${JWT_SECRET}  # Base64-encoded 256-bit key
  access-token-expiration: 900000   # 15 minutes
  refresh-token-expiration: 604800000  # 7 days

server:
  port: 8080
```

Part 16: Build It Yourself -- JWT Authentication System

> ***Goal:*** *Build a complete Spring Security JWT authentication system from scratch -- registration, login, token issuance, and role-based access control.*

Concept Overview

```
Registration:  POST /api/auth/register -> Save user (BCrypt password) -> Return
success
Login:         POST /api/auth/login    -> Validate credentials -> Return JWT
API Call:      GET /api/users/me       -> JWT in header -> JwtFilter validates -
> Return data
```

Component	Purpose
`SecurityConfig`	Defines which URLs need auth, which are public
`JwtService`	Creates and validates JWT tokens
`JwtAuthenticationFilter`	Intercepts every request, extracts and validates JWT
`CustomUserDetailsService`	Loads user from database for Spring Security
`AuthController`	Login and register endpoints

Step 1: Add Dependencies

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.5</version>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.5</version>
    <scope>runtime</scope>
</dependency>
```

Step 2: Create JwtService

Concept: JWT = Header.Payload.Signature. The server signs the token with a secret key. On each request, it validates the signature -- no database lookup needed.

```
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    public String generateToken(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claim("roles", user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 900_000))  // 15
min
            .signWith(getSigningKey())
            .compact();
    }

    public String extractUsername(String token) {
        return getClaims(token).getSubject();
    }

    public boolean isTokenValid(String token, UserDetails user) {
        return extractUsername(token).equals(user.getUsername())
            && !getClaims(token).getExpiration().before(new Date());
    }
```

```
    private Claims getClaims(String token) {
        return Jwts.parser().verifyWith(getSigningKey())
            .build().parseSignedClaims(token).getPayload();
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretKey));
    }
}
```

Step 3: Create JWT Authentication Filter

Concept: This filter runs BEFORE every request. It extracts the JWT from the `Authorization` header, validates it, and sets the authenticated user in Spring's SecurityContext.

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);       // No token -> continue
(might be public URL)
            return;
        }

        String token = authHeader.substring(7);
        String username = jwtService.extractUsername(token);

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails user = userDetailsService.loadUserByUsername(username);
            if (jwtService.isTokenValid(token, user)) {
                UsernamePasswordAuthenticationToken auth =
```

```
                    new UsernamePasswordAuthenticationToken(user, null,
user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(request, response);
    }
}
```

Step 4: Configure Security

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired private JwtAuthenticationFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        return http
            .csrf(csrf -> csrf.disable())              // Disable CSRF for REST
APIs
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))  // No
sessions
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()   // Public:
login, register
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,                // Add JWT filter to
chain
                UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);          // BCrypt with strength
12
```

```
    }
}
```

Step 5: Create Auth Controller

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtService jwtService;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @PostMapping("/register")
    public ResponseEntity<String> register(@RequestBody RegisterRequest req) {
        User user = new User();
        user.setEmail(req.email());
        user.setPasswordHash(passwordEncoder.encode(req.password()));
        user.setRole(Role.USER);
        userRepository.save(user);
        return ResponseEntity.status(201).body("User registered");
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest req) {
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.email(),
req.password()));
        UserDetails user = userDetailsService.loadUserByUsername(req.email());
        String token = jwtService.generateToken(user);
        return ResponseEntity.ok(new AuthResponse(token));
    }
}
```

Step 6: Test the Flow

```
# Register
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"user@test.com","password":"secret123"}'

# Login (get JWT)
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@test.com","password":"secret123"}'
# Response: {"token":"eyJhbGciOiJIUzI1NiIs..."}

# Access protected endpoint
curl http://localhost:8080/api/users/me \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```

Key Takeaways

Concept	Remember
JWT is stateless	Server doesn't store sessions -- token contains all user info
BCrypt for passwords	Never store plain text; use strength 10-12
Filter chain order	JwtFilter BEFORE UsernamePasswordAuthenticationFilter
`@PreAuthorize`	Method-level security: `@PreAuthorize("hasRole('ADMIN')")`
Token expiry	Access = 15 min, Refresh = 7 days, always short-lived

---

Part 16: Spring Security Interview Questions (100+)

Fundamentals (Q1-Q20)

Q1. What is Spring Security? A powerful, customizable authentication and access-control framework for Spring-based applications. Provides comprehensive security services for Java EE applications including authentication, authorization, and protection against common exploits.

Q2. What is the difference between authentication and authorization? Authentication (AuthN): Verifying identity ("Who are you?"). Authorization (AuthZ): Verifying permissions ("What can you do?"). Authentication happens first; authorization happens after successful authentication.

Q3. What is the SecurityFilterChain? A chain of servlet filters that process every incoming HTTP request. Each filter handles a specific security concern (CSRF, authentication, authorization). Configured via `HttpSecurity` in a `@Configuration` class.

Q4. What is SecurityContextHolder? A holder class that stores the `SecurityContext`, which contains the `Authentication` object of the currently authenticated user. Uses ThreadLocal by default, so authentication is available throughout the request lifecycle.

Q5. What is the default authentication mechanism in Spring Security? Form-based login with a generated password. Spring Boot auto-generates a password on startup and provides a default login page at `/login`.

Q6. How does Spring Security store passwords? Using `PasswordEncoder`. The recommended encoder is `BCryptPasswordEncoder`, which uses BCrypt hashing with a random salt. Passwords are never stored as plain text.

Q7. What is CSRF and how does Spring Security handle it? Cross-Site Request Forgery: an attacker tricks a user's browser into making unwanted requests. Spring Security includes a CSRF token in forms and validates it on submission. Disabled for stateless REST APIs (JWT-based).

Q8. What is CORS? Cross-Origin Resource Sharing: a browser mechanism that allows/restricts requests from different origins. Configure in Spring Security with `CorsConfigurationSource` to whitelist allowed origins, methods, and headers.

Q9. What is the difference between `@Secured` and `@PreAuthorize`? `@Secured`: simple role check, no SpEL support (`@Secured("ROLE_ADMIN")`). `@PreAuthorize`: supports Spring

Expression Language for complex conditions (`@PreAuthorize("hasRole('ADMIN') and #id == authentication.principal.id")`). `@PreAuthorize` is more powerful and recommended.

Q10. What is the `UserDetailsService` interface? Core interface that loads user-specific data. Has a single method: `loadUserByUsername(String username)` that returns a `UserDetails` object. Implementation typically queries the database.

Q11. What is the `UserDetails` interface? Represents the authenticated user. Contains: username, password, authorities, and account status flags (expired, locked, enabled). Custom implementations wrap your domain User entity.

Q12. What is `GrantedAuthority`? Represents a permission/role granted to the user. Stored as a string (e.g., "ROLE_ADMIN", "user:read"). Retrieved from `Authentication.getAuthorities()`.

Q13. What is the `AuthenticationManager`? Central interface for authentication. Receives an `Authentication` request and returns a fully populated `Authentication` if successful. Default implementation is `ProviderManager`.

Q14. What is an `AuthenticationProvider`? Performs the actual authentication logic. The `AuthenticationManager` delegates to one or more providers. `DaoAuthenticationProvider` is the most common (uses `UserDetailsService` + `PasswordEncoder`).

Q15. How to disable Spring Security for certain endpoints? Use `.requestMatchers("/public/**").permitAll()` in the `SecurityFilterChain` configuration. This allows unauthenticated access to matching URLs.

Q16. What is session fixation attack? An attacker sets a known session ID on the victim's browser before they log in. After login, the attacker uses the same session ID to hijack the session. Spring Security mitigates this by creating a new session on authentication.

Q17. What is `DelegatingPasswordEncoder`? A `PasswordEncoder` that supports multiple encoding formats. Prefixes the hash with the algorithm: `{bcrypt}$2a$10$...`. Useful for migrating from one hashing algorithm to another.

Q18. What is method-level security? Securing individual methods with annotations. Enabled with `@EnableMethodSecurity`. Annotations: `@PreAuthorize` (before), `@PostAuthorize` (after), `@Secured`, `@RolesAllowed`.

Q19. What HTTP status codes does Spring Security return? 401 Unauthorized: authentication required or failed. 403 Forbidden: authenticated but insufficient permissions. These are handled by `AuthenticationEntryPoint` and `AccessDeniedHandler`.

Q20. What is the difference between `hasRole` and `hasAuthority`? `hasRole("ADMIN")` checks for `ROLE_ADMIN` (auto-prefixes "ROLE_"). `hasAuthority("ROLE_ADMIN")` checks for the exact string. `hasAuthority` is also used for fine-grained permissions like `user:read`.

JWT and Token Security (Q21-Q40)

Q21. What is JWT? JSON Web Token: a compact, self-contained token format (Header.Payload.Signature). Contains user claims, signed with a secret key. Used for stateless authentication in REST APIs.

Q22. What are the three parts of a JWT? Header (algorithm + type), Payload (claims: subject, roles, expiration), Signature (HMAC or RSA signature for integrity verification).

Q23. Where should JWT be stored on the client? HttpOnly cookies (most secure, immune to XSS). localStorage is convenient but vulnerable to XSS attacks. Never store in URL parameters.

Q24. What is the difference between access token and refresh token? Access token: short-lived (15 min), used for API authentication. Refresh token: long-lived (7 days), used only to obtain new access tokens. Refresh tokens should be stored securely and can be revoked.

Q25. How to revoke a JWT? JWTs are self-contained and can't be modified after issuance. Options: 1) Short expiry (damage window limited). 2) Token blacklist in Redis. 3) Token versioning (increment version on logout). 4) Refresh token rotation.

Q26. What is token rotation? When a refresh token is used, issue both a new access token AND a new refresh token. Invalidate the old refresh token. If the old token is reused, revoke all tokens for that user (indicates theft).

Q27. What signing algorithms are used for JWT? HS256 (HMAC-SHA256): symmetric, same key for signing and verification. RS256 (RSA-SHA256): asymmetric, private key signs, public key verifies. RS256 is preferred for distributed systems (microservices can verify without the private key).

Q28. How does JWT authentication work in Spring Security? Custom `OncePerRequestFilter` extracts JWT from Authorization header, validates signature, extracts username, loads `UserDetails`, creates `Authentication` token, and sets it in `SecurityContextHolder`.

Q29. What happens when a JWT expires? The server returns 401. The client uses its refresh token to obtain a new access token from the `/auth/refresh` endpoint. If the refresh token is also expired, the user must re-login.

Q30. What is the `jti` claim in JWT? JWT ID: a unique identifier for the token. Used to prevent replay attacks and implement token revocation (blacklist by jti instead of the entire token string).

Q31. How to implement "logout" with JWT? Since JWT is stateless: 1) Client deletes the token. 2) Add token to server-side blacklist (Redis) with TTL equal to remaining expiry time. 3) Invalidate the refresh token in database.

Q32. What is the difference between symmetric and asymmetric JWT signing? Symmetric (HMAC): same secret key for signing and verifying. Simpler but secret must be shared with all validators. Asymmetric (RSA/ECDSA): private key signs, public key verifies. More secure for distributed systems.

Q33. How to handle JWT in a microservices architecture? Auth Service generates JWT. API Gateway validates JWT on every request. Downstream services either trust the gateway (read user headers) or re-validate the JWT.

Q34. What is a JWK (JSON Web Key)? A JSON representation of a cryptographic key. Used with asymmetric JWT. Auth server publishes public keys at a JWK endpoint. Resource servers fetch keys to verify tokens.

Q35. What security risks does JWT have? Token theft (XSS/network), no server-side revocation, payload is only Base64 encoded (not encrypted), algorithm confusion attacks. Mitigate with short expiry, HttpOnly cookies, HTTPS, and explicit algorithm verification.

Q36. What is the difference between JWT and opaque tokens? JWT: self-contained, no server lookup needed, larger size. Opaque: random string, requires server lookup (introspection), smaller size, easier revocation. OAuth2 can use either.

Q37. How to pass additional information in JWT? Add custom claims to the payload: roles, permissions, tenant ID, organization. Keep payload small to minimize token size. Never include sensitive data (passwords, SSN).

Q38. What is token introspection? An OAuth2 endpoint where resource servers can verify if an opaque token is active. The auth server returns token metadata (active, scope, expiry). Used when tokens are not self-contained.

Q39. How to test JWT authentication? Use `@WithMockUser` for unit tests. For integration tests, generate a valid JWT in the test setup. Use `SecurityMockMvcRequestPostProcessors.jwt()` with Spring Security test support.

Q40. What is the `nbf` (Not Before) claim? Specifies the time before which the token must not be accepted. Used to issue tokens for future use. If current time < nbf, token is rejected.

OAuth2 (Q41-Q55)

Q41. What is OAuth2? An authorization framework that allows third-party applications to obtain limited access to a user's resources without sharing credentials. Delegates authentication to an authorization server.

Q42. What are the four OAuth2 roles? Resource Owner (user), Client (application), Authorization Server (issues tokens), Resource Server (hosts protected resources).

Q43. What is the Authorization Code flow? Most secure flow for web apps. User is redirected to auth server, logs in, auth server returns authorization code, client exchanges code for tokens server-side. The code is short-lived and single-use.

Q44. What is the Client Credentials flow? Machine-to-machine authentication. Client sends its own credentials (client_id + client_secret) directly to get an access token. No user involvement. Used for service-to-service communication.

Q45. What is the difference between OAuth2 and OpenID Connect? OAuth2 is for authorization (accessing resources). OpenID Connect (OIDC) is an identity layer on top of OAuth2 for authentication (knowing who the user is). OIDC adds an ID token with user information.

Q46. What is an ID token vs access token? ID token: contains user identity information (name, email), intended for the client app. Access token: used to access protected APIs, intended for the resource server. ID tokens are JWT; access tokens can be opaque.

Q47. What is a scope in OAuth2? Defines the level of access requested. Examples: `openid`, `profile`, `email`, `read:orders`. The user must consent to the requested scopes. The auth server includes granted scopes in the token.

Q48. What is Spring Authorization Server? A framework for building OAuth2 Authorization Servers with Spring Security. Supports authorization code, client credentials, refresh token, and device authorization grants.

Q49. What is Keycloak? An open-source identity and access management solution. Provides OAuth2, OIDC, SAML, and social login out of the box. Commonly used as the authorization server in Spring microservices.

Q50. How to configure Spring Boot as an OAuth2 Resource Server?

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8080/realms/myapp
```

This auto-configures JWT validation using the issuer's public keys.

Q51. What is PKCE (Proof Key for Code Exchange)? An extension to the Authorization Code flow for public clients (mobile/SPA). The client generates a code_verifier, sends a code_challenge to the auth server, and proves possession of the verifier when exchanging the code.

Q52. What is token exchange? An OAuth2 extension where a service exchanges one token for another with different scope or audience. Used in microservices when Service A needs to call Service B on behalf of the user.

Q53. What is the refresh token grant? Uses a refresh token to obtain a new access token without re-authentication. The auth server validates the refresh token and issues a new access token (and optionally a new refresh token).

Q54. How does "Login with Google" work in Spring Boot? Configure Google as an OAuth2 client in application.yml. Spring Security redirects to Google's consent page. After consent, Google redirects back with a code. Spring exchanges the code for tokens and loads user info.

Q55. What is multi-tenancy in OAuth2? Supporting multiple organizations/tenants in one application. Each tenant may have its own auth server or realm. Implemented by dynamically resolving the issuer based on the request.

Architecture and Production (Q56-Q80)

Q56. How to implement RBAC in Spring Security? Define roles in database. Map roles to authorities. Use `@PreAuthorize("hasRole('ADMIN')")` for method security and `.hasRole("ADMIN")` for URL security. Load authorities in `UserDetailsService`.

Q57. What is the filter chain execution order? SecurityContextPersistenceFilter -> CsrfFilter -> LogoutFilter -> AuthenticationFilter -> ExceptionTranslationFilter -> AuthorizationFilter. Custom filters can be added before/after any filter.

Q58. How to configure multiple SecurityFilterChains? Define multiple `@Bean` `SecurityFilterChain` methods with `@Order`. Lower order = higher priority. Use

`securityMatcher` to scope each chain to specific URLs.

Q59. What is the `OncePerRequestFilter`? A filter base class that guarantees a single execution per request. Used for custom filters like JWT authentication. Prevents duplicate filter execution in forwarded requests.

Q60. How to secure WebSocket connections? Configure `WebSocketSecurityConfigurer`. Authenticate during the WebSocket handshake. Use STOMP message-level authorization for individual message types.

Q61. How to implement password reset flow? User requests reset, generate a time-limited token (UUID), store hash in DB, email the link. On reset, validate token, hash new password with BCrypt, invalidate the token.

Q62. How to implement account lockout? Track failed login attempts per user. After N failures, set `locked = true`. Unlock after a timeout or manual admin action. Check `isAccountNonLocked()` in `UserDetails`.

Q63. What is the `ExceptionTranslationFilter`? Catches security exceptions. Translates `AuthenticationException` to 401 (via `AuthenticationEntryPoint`). Translates `AccessDeniedException` to 403 (via `AccessDeniedHandler`).

Q64. How to implement remember-me authentication? `http.rememberMe()` generates a persistent token stored in a cookie. Spring validates the token on subsequent requests. Use `PersistentTokenBasedRememberMeServices` for production (tokens stored in DB).

Q65. How to secure actuator endpoints? `.requestMatchers("/actuator/**").hasRole("ADMIN")` or restrict to specific endpoints. In production, expose only health and info endpoints. Use separate security chain with higher priority.

Q66. What is HSTS (HTTP Strict Transport Security)? A security header that tells browsers to always use HTTPS. Spring Security adds this by default. Prevents SSL stripping attacks. Configuration: `headers.httpStrictTransportSecurity()`.

Q67. How to prevent brute force attacks? Rate limiting per IP/user. Account lockout after N failures. CAPTCHA after repeated failures. Login attempt logging and alerting. Exponential backoff on failed attempts.

Q68. What is content negotiation in security? Returning different error formats based on the Accept header. For browser requests: redirect to login page. For API requests: return JSON 401 error. Configured in `AuthenticationEntryPoint`.

Q69. How to implement audit logging for security events? Listen to Spring Security events: `AuthenticationSuccessEvent`, `AuthenticationFailureBadCredentialsEvent`, `AuthorizationDeniedEvent`. Use `@EventListener` to log to database or monitoring system.

Q70. What is the `SecurityContextRepository`? Interface that controls how `SecurityContext` is stored between requests. `HttpSessionSecurityContextRepository` (default) stores in session. For stateless: use `RequestAttributeSecurityContextRepository`.

Q71. How to test Spring Security? Use `@WithMockUser(roles = "ADMIN")` for mock authentication. Use `SecurityMockMvcRequestPostProcessors` for MockMvc tests. Use `@SpringBootTest` for integration tests with real security config.

Q72. What is the `AbstractSecurityInterceptor`? Base class that performs authorization checks. Intercepts method/URL access, uses `AccessDecisionManager` to vote on whether access should be granted. Replaced by `AuthorizationManager` in Spring Security 6.

Q73. How to configure security for different API versions? Use `securityMatcher("/api/v1/**")` and `securityMatcher("/api/v2/**")` on separate `SecurityFilterChain` beans with `@Order`. Each version can have different authentication requirements.

Q74. How to implement IP-based access control? Use `hasIpAddress("192.168.1.0/24")` in the security config. Or create a custom filter that checks request IP against an allowlist. Useful for admin endpoints.

Q75. What is the `AuthenticationSuccessHandler`? Invoked after successful authentication. Customizes post-login behavior: redirect URL, response body (JWT), audit logging, session attribute setup.

Q76. How to secure file uploads? Validate file type (whitelist extensions), check file size, scan for malware, store outside web root, generate random filenames, set Content-Disposition header on download.

Q77. What is Spring Security ACL? Access Control List: domain object security. Controls access to individual objects (e.g., "User 1 can edit Document 42 but not Document 43"). More granular than role-based security.

Q78. How to implement API key authentication? Create a custom filter that reads the API key from a header (`X-API-Key`). Validate against stored keys in database. Create an `Authentication`

object for the API key owner.

Q79. What is mTLS (Mutual TLS)? Both client and server present X.509 certificates. The server validates the client's certificate. Used for service-to-service authentication in microservices. Configured in Spring with `spring.ssl.bundles`.

Q80. How to rotate JWT signing keys? Use key identifiers (`kid` in JWT header). Publish multiple keys at the JWK endpoint. Sign new tokens with the new key. Verify tokens using the `kid` to select the correct key. Remove old keys after their tokens expire.

Advanced & Architect-Level (Q81-Q105)

Q81. Design a security architecture for a microservices system. API Gateway validates JWT. Auth Service issues tokens. Each service extracts user from JWT/headers. RBAC at service level. mTLS for service-to-service. Secrets in Vault. Rate limiting at Gateway.

Q82. How to implement SSO across multiple applications? Central auth server (Keycloak). All apps redirect to the same auth server. Session cookie at the auth server domain. Once logged in to one app, the auth server recognizes the session for other apps.

Q83. Design a multi-tenant security system. Tenant ID in JWT claims. Tenant resolver extracts tenant from request (subdomain, header, JWT). Data isolation: tenant column in tables or separate databases. Security filter adds tenant context.

Q84. How to implement field-level security? Use `@PostFilter` to filter collection results. Use `@JsonView` with security-aware views. Or use DTO mappers that check permissions before including sensitive fields.

Q85. What is the principle of least privilege? Users and services should have only the minimum permissions needed. Default deny all, explicitly grant. Regular permission audits. Avoid wildcard permissions.

Q86. How to implement dynamic authorization rules? Store rules in database. Custom `AuthorizationManager` loads rules at runtime. Update rules without redeployment. Cache rules with TTL for performance.

Q87. How to secure event-driven systems? Sign Kafka messages with producer credentials. Validate consumer authorization. Encrypt sensitive payloads. Audit message processing. Use SASL/SSL for Kafka connections.

Q88. What is zero trust security? Never trust, always verify. Every request is authenticated and authorized regardless of network location. No implicit trust for internal services. mTLS everywhere. Continuous validation.

Q89. How to implement consent management? Track what data the user has consented to share. Store consent records per user per scope. Check consent before returning data. Provide consent revocation API. GDPR compliance.

Q90. Design a secure password reset flow.

1. User enters email. 2. Generate cryptographically random token. 3. Store token hash in DB with 15-min expiry. 4. Email link with token. 5. User clicks link, enters new password. 6. Validate token, hash new password, delete token. 7. Invalidate all existing sessions.

Q91. How to implement security in GraphQL? Authenticate at the HTTP level (JWT filter). Authorize at the resolver level using `@PreAuthorize` or custom directives. Limit query depth and complexity to prevent DoS.

Q92. What security considerations for WebSockets? Authenticate during handshake. Validate authorization per message. Rate limit messages. Validate message content. Close idle connections. Prevent CSWSH (Cross-Site WebSocket Hijacking).

Q93. How to implement security headers best practices? Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Referrer-Policy, Permissions-Policy. Spring Security's `headers()` configures most automatically.

Q94. How to handle security in blue-green deployments? Both versions must accept the same JWT signing key. Share the same auth server. Token blacklists must be shared (Redis). Session stores must be shared during transition.

Q95. Design an audit logging system for security events. Log: who (user), what (action), when (timestamp), where (IP, service), result (success/failure). Store in append-only database. Tamper detection with checksums. Retention policy per compliance requirements.

Q96. How to implement device fingerprinting? Collect device attributes (browser, OS, screen, timezone). Hash into a fingerprint. Store known devices per user. Alert on unknown device login. Require MFA for new devices.

Q97. What is a security token service (STS)? A service that issues, validates, and renews security tokens. Central point for token management. Implements OAuth2 Authorization Server. Handles token lifecycle.

Q98. How to implement data encryption at rest? Database-level encryption (TDE). Application-level field encryption for sensitive fields (SSN, credit card). Use JCA/JCE for encryption. Manage keys in Vault or KMS.

Q99. How to secure configuration and secrets? Never hardcode secrets. Use environment variables or Vault. Encrypt application.yml sensitive values with Jasypt. Rotate secrets regularly. Use Kubernetes Secrets with encryption at rest.

Q100. What is the OWASP Top 10 and how does Spring Security address it? Injection (parameterized queries), Broken Auth (Spring Security framework), Sensitive Data Exposure (HTTPS/encryption), XXE (disable external entities), Broken Access Control (RBAC), Security Misconfig (secure defaults), XSS (content security headers), Insecure Deserialization (type validation), Components with Vulnerabilities (dependency scanning), Insufficient Logging (audit events).

Q101. How to implement step-up authentication? Allow basic access with standard auth. For sensitive operations (transfer money), require additional authentication (password re-entry, MFA). Implement as a custom filter or `@PreAuthorize` check.

Q102. Design a centralized authorization service. Standalone service managing all permissions. Expose API: `POST /authorize { userId, resource, action }`. Decision caching at service level. Policy-based rules (OPA/Casbin). Event-driven policy updates.

Q103. How to implement API versioning with security? Each API version can have different security requirements. Use separate `SecurityFilterChain` per version. Deprecate authentication methods with version migration. Support backwards compatibility.

Q104. What is threat modeling? Systematic process to identify and address security threats. Techniques: STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). Apply to each microservice.

Q105. How to prepare for a security audit? Document authentication flows. Map all API endpoints with their authorization requirements. Verify password policies. Review dependency vulnerabilities. Check compliance requirements (GDPR, PCI-DSS). Provide audit logs.

Part 17: Hands-on Practice Exercises

Exercise 1: Build JWT Authentication System

Goal: Create a complete JWT auth system from scratch.

```
// Step 1: User entity + Role entity
// Step 2: UserRepository + RoleRepository
// Step 3: CustomUserDetails + CustomUserDetailsService
// Step 4: JwtService (generate/validate tokens)
// Step 5: JwtAuthenticationFilter
// Step 6: SecurityConfig
// Step 7: AuthController (login/register/refresh)
// Step 8: Test with Postman

// Expected endpoints:
// POST /api/auth/register -> 201 + tokens
// POST /api/auth/login    -> 200 + tokens
// POST /api/auth/refresh  -> 200 + new access token
// GET  /api/users/me      -> 200 + user data (requires JWT)
```

Exercise 2: Implement OAuth2 Login with Google

```yaml
# application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: openid, profile, email
```

```
// Step 1: Register app in Google Developer Console
// Step 2: Configure application.yml
// Step 3: Create CustomOAuth2UserService
```

```
// Step 4: Create OAuth2 success handler (generate JWT after Google login)
// Step 5: Test: visit /oauth2/authorization/google
```

Exercise 3: Implement Role-Based Authorization

```
// Step 1: Create roles table with ADMIN, USER, MANAGER
// Step 2: Assign roles to users via user_roles table
// Step 3: Configure URL-based security:

.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/reports/**").hasAnyRole("ADMIN", "MANAGER")
.requestMatchers("/api/profile/**").authenticated()

// Step 4: Configure method-level security:
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long userId) { ... }

@PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
public User getUser(Long userId) { ... }

// Step 5: Test each role accessing each endpoint
```

Exercise 4: Implement Refresh Token Rotation

```
// Step 1: Create RefreshToken entity with: token, userId, expiryDate, revoked
// Step 2: On login: generate access + refresh tokens, store refresh in DB
// Step 3: On refresh:
//   a. Validate refresh token exists and not expired
//   b. Generate new access token + new refresh token
//   c. Revoke old refresh token
//   d. Return new tokens
// Step 4: On logout:
//   Revoke all refresh tokens for the user

@Entity
public class RefreshToken {
    @Id @GeneratedValue
    private Long id;
    private String token;
    private Long userId;
```

```
    private Instant expiryDate;
    private boolean revoked;
}
```

Exercise 5: Build Rate Limiting Filter

```
// Step 1: Create a filter that extends OncePerRequestFilter
// Step 2: Track requests per IP using a ConcurrentHashMap + timestamps
// Step 3: Allow max 100 requests per minute per IP
// Step 4: Return 429 Too Many Requests when limit exceeded
// Step 5: Register filter in SecurityFilterChain
// Step 6: Test with rapid API calls

@Component
public class RateLimitFilter extends OncePerRequestFilter {
    private final Map<String, List<Long>> requests = new ConcurrentHashMap<>();

    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String ip = req.getRemoteAddr();
        long now = System.currentTimeMillis();
        requests.computeIfAbsent(ip, k -> new ArrayList<>())
                .removeIf(t -> now - t > 60000);
        if (requests.get(ip).size() >= 100) {
            res.setStatus(429);
            res.getWriter().write("{\"error\":\"Rate limit exceeded\"}");
            return;
        }
        requests.get(ip).add(now);
        chain.doFilter(req, res);
    }
}
```

Exercise 6: Implement Account Lockout

```
// Step 1: Add loginAttempts and lockedUntil fields to User entity
// Step 2: On failed login: increment loginAttempts
// Step 3: After 5 failures: set lockedUntil = now + 30 minutes
```

```
// Step 4: On login attempt: check if account is locked
// Step 5: On successful login: reset loginAttempts

@EventListener
public void onAuthFailure(AuthenticationFailureBadCredentialsEvent event) {
    String email = (String) event.getAuthentication().getPrincipal();
    User user = userRepository.findByEmail(email).orElse(null);
    if (user != null) {
        user.setLoginAttempts(user.getLoginAttempts() + 1);
        if (user.getLoginAttempts() >= 5) {
            user.setLockedUntil(LocalDateTime.now().plusMinutes(30));
        }
        userRepository.save(user);
    }
}
```

Exercise 7: Secure Microservices Architecture

```
// Step 1: Create Auth Service (issues JWT)
// Step 2: Create API Gateway (validates JWT, routes requests)
// Step 3: Create Order Service (reads user from headers)
// Step 4: Create User Service (RBAC on endpoints)
// Step 5: Implement Feign client that propagates JWT
// Step 6: Test end-to-end flow:
//   Login -> Get JWT -> Call Order Service via Gateway
//   -> Order Service calls User Service with propagated JWT
```

Exercise 8: Implement Audit Logging

```
// Step 1: Create AuditLog entity (user, action, ip, timestamp, result)
// Step 2: Listen to Spring Security events:

@Component
public class SecurityAuditListener {
    @Autowired private AuditLogRepository auditRepo;

    @EventListener
    public void onSuccess(AuthenticationSuccessEvent event) {
        auditRepo.save(new AuditLog(
            event.getAuthentication().getName(),
            "LOGIN_SUCCESS", getClientIp(), LocalDateTime.now()));
```

```
    }

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent event) {
        auditRepo.save(new AuditLog(
            (String) event.getAuthentication().getPrincipal(),
            "LOGIN_FAILED", getClientIp(), LocalDateTime.now()));
    }
}
```

Exercise 9: Implement CORS for Multiple Frontends

```
// Step 1: Configure allowed origins for different environments
// Step 2: Allow specific methods (GET, POST, PUT, DELETE)
// Step 3: Allow Authorization and Content-Type headers
// Step 4: Test from different origins

@Bean
public CorsConfigurationSource corsConfig() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of(
        "http://localhost:3000",       // Dev frontend
        "https://app.mycompany.com",   // Production
        "https://admin.mycompany.com"  // Admin portal
    ));
    config.setAllowedMethods(List.of("GET","POST","PUT","DELETE","OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization","Content-Type"));
    config.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}
```

Exercise 10: Implement Custom Authentication Provider

```
// Step 1: Create a custom AuthenticationProvider
// Step 2: Authenticate against an external API or custom logic
// Step 3: Register in SecurityConfig
```

```java
@Component
public class ApiKeyAuthProvider implements AuthenticationProvider {
    @Autowired private ApiKeyRepository apiKeyRepo;

    @Override
    public Authentication authenticate(Authentication auth)
            throws AuthenticationException {
        String apiKey = (String) auth.getCredentials();
        ApiKeyEntity key = apiKeyRepo.findByKeyValue(apiKey)
            .orElseThrow(() -> new BadCredentialsException("Invalid API key"));

        if (!key.isActive() || key.getExpiresAt().isBefore(Instant.now())) {
            throw new BadCredentialsException("API key expired");
        }

        return new UsernamePasswordAuthenticationToken(
            key.getOwner(), null,
            List.of(new SimpleGrantedAuthority("ROLE_API_CLIENT")));
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return
ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

End of Guide

Document Version: 1.0 **Last Updated:** March 2026 **Topics Covered:** 17 Parts, 105 Interview Questions, 10 Hands-on Exercises